Cloud-Based Applications Assignment 1: Country Lookup REST Service

University: University of Luxembourg
Course: Cloud-Based Applications
Student: Eman Abdai
Programme: BINFO

For this assignment, I developed a Spring Boot REST service that returns the country and country code for a given latitude and longitude. I also created a multi-stage Dockerfile and deployed the application in Minikube with two replicas.

I worked around my laptop's locked BIOS settings by using GitHub Codespaces to run Docker and Minikube and access the service through a forwarded port.

Deliverables

- Spring Boot source code and Maven build file.
- Multi-stage `Dockerfile`.
- Kubernetes manifest: `deployment.yaml`.
- README in Markdown and PDF.
- Unlisted YouTube video link:https://youtu.be/3M4Fi0_HHJo

Prerequisites

The following tools are needed:

- Java 17 and Maven.
- Docker.
- Minikube and kubectl.
- Internet access for the OpenStreetMap Nominatim API.

Run the commands below from the project root folder. Make sure port 8080 is available. Stop any locally running application or Docker container using that port before starting Kubernetes port forwarding.

Part 1: Spring Boot REST Service

How It Works

The endpoint `/api/country` accepts two query parameters:

- `latitude`: a number between -90 and 90, inclusive.
- `longitude`: a number between -180 and 180, inclusive.

The application uses Spring `WebClient` to request country information from the OpenStreetMap Nominatim API.

Missing, blank, nonnumeric, or out-of-range coordinates return **400 Bad Request**. If the external API fails or returns unusable data, the service returns **502 Bad Gateway**.

Build and Run

Build the application:

```
mvn -f demo/pom.xml clean package
```

Start the application:

```
mvn -f demo/pom.xml spring-boot:run
```

Access the Service

Open this URL in a browser:

```
http://localhost:8080/api/country?latitude=45.0&longitude=9.0
```

Or use curl:

```
curl "http://localhost:8080/api/country?latitude=45.0&longitude=9.0"
```

Expected response:

```
{
  "country": "Italy",
```

```
  "countryCode": "IT",
  "attribution": "Data © OpenStreetMap contributors, ODbL 1.0: https://www.openstreet
}
```

In GitHub Codespaces, open port 8080 from the **Ports** tab and append `/api/country?latitude=45.0&longitude=9.0` to the forwarded URL.

Check Invalid Inputs

These requests should return HTTP 400.

Missing latitude:

```
curl -i "http://localhost:8080/api/country?longitude=9.0"
```

Latitude outside the allowed range:

```
curl -i "http://localhost:8080/api/country?latitude=91&longitude=9.0"
```

Longitude outside the allowed range:

```
curl -i "http://localhost:8080/api/country?latitude=45.0&longitude=181"
```

Nonnumeric latitude:

```
curl -i "http://localhost:8080/api/country?latitude=abc&longitude=9.0"
```

Blank latitude:

```
curl -i "http://localhost:8080/api/country?latitude=&longitude=9.0"
```

Part 2: Docker

The Dockerfile uses two stages. The first stage builds the application with Maven and Eclipse Temurin 17. The second stage runs the compiled application using an Alpine JRE, without including Maven or the source code.

Build the Image

```
docker build -t geolocator-service:latest -f Dockerfile .
```

Run the Container

```
docker run --rm -p 8080:8080 geolocator-service:latest
```

The service can then be accessed using the same URL shown in Part 1.

Press **Ctrl+C** to stop the container before using port 8080 for Kubernetes port forwarding.

Image Size

I checked the image size with:

```
docker images geolocator-service:latest
```

Docker reported:

- **Disk usage:** 338 MB.
- **Content size:** 108 MB.

Part 3: Kubernetes with Minikube

The `deployment.yaml` file defines a Deployment with **two replicas** and a ClusterIP Service.

1. Start Minikube

```
minikube start --driver=docker
```

2. Build the Image

If the image has not already been built:

```
docker build -t geolocator-service:latest -f Dockerfile .
```

3. Load the Image into Minikube

```
minikube image load geolocator-service:latest
```

4. Apply the Manifest

```
kubectl apply -f deployment.yaml
```

5. Wait for the Deployment

```
kubectl rollout status deployment/country-lookup --timeout=180s
```

Check the pods:

```
kubectl get pods
```

Both application pods should show **1/1** under `READY` and **Running** under `STATUS`.

6. Start Port Forwarding

```
kubectl port-forward svc/country-lookup 8080:8080
```

Leave this terminal running.

7. Open the Service in a Browser

```
http://localhost:8080/api/country?latitude=45.0&longitude=9.0
```

In GitHub Codespaces, select **Ports** → **8080** → **Open in Browser**, then append `/api/country?latitude=45.0&longitude=9.0` to the forwarded URL.

The response should contain `Italy` and `IT`.

The service can also be checked from a separate terminal:

```
curl "http://localhost:8080/api/country?latitude=45.0&longitude=9.0"
```

Updating the Application

After rebuilding the Docker image, run:

```
minikube image load geolocator-service:latest
kubectl rollout restart deployment/country-lookup
kubectl rollout status deployment/country-lookup --timeout=180s
kubectl get pods
```

After the rollout finishes, stop any existing port forwarding with **Ctrl+C** and start it again:

```
kubectl port-forward svc/country-lookup 8080:8080
```

Working Status and Limitations

I successfully started Minikube, built the Docker image, loaded it into the cluster, applied the Kubernetes manifest, and completed the deployment rollout. Both application pods showed **1/1 Running**. I also accessed the service in a browser and received the response containing **Italy** and **IT**.

The application needs an internet connection and depends on the external Nominatim API. Port forwarding must remain running while accessing the Kubernetes service.

The successful browser request confirms that the normal lookup works. The screencast does not demonstrate every validation or external API error case.

Screencast

The video shows Minikube startup, the Docker image build, loading the image into Minikube, applying the Kubernetes manifest, restarting the deployment, checking two running replicas, and accessing the service in a browser.

Unlisted YouTube video link:https://youtu.be/3M4Fi0_HHJo

AI Assistance

I used AI assistance to help with the implementation, answer questions, and documentation.